Python, Properly.

Day 1 we installed everything. Today we write code that earns its place in a real project — functions that return things, files that remember things.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

2 of 24

AGENDA · DAY 02

How we'll spend our 90 minutes today.

More keyboard, fewer slides. Today is about writing functions that actually do something, and getting files to remember data between runs.

DURATION
90 min

00:00 – 00:10

Day 1 Recap & Q&A

Tools check, install fixes from yesterday, open floor for questions.

00:10 – 00:25

Functions With Real Inputs

Beyond print — functions that take args and return values other code can use.

00:25 – 00:40

Reading & Writing Files

open, read, write, with-blocks. The foundation of every project from Day 9 onward.

00:40 – 00:55

Pitfalls & try/except

Mutable default args, == vs is, file-not-found, and how Python handles errors.

00:55 – 01:15

Closing Exercise: Contact List Cleaner

Build a function that reads a messy contacts.txt and returns clean (name, email) tuples.

01:15 – 01:30

Q&A and Day 3 Preview

Open floor, then a quick look at Git & GitHub.

RECAP · 10 MIN

Yesterday we set the table. Today we cook.

Quick check that every laptop is ready, then we move on. No blame if something broke overnight — we fix it and move.

WHAT WE COVERED YESTERDAY

- ✓ Installed Python, VS Code, Git, Streamlit on every laptop
- ✓ Set up a virtual environment (`.venv`) per project
- ✓ Ran `streamlit hello` and saw it open in a browser
- ✓ Built a 5-line hello-world app and ran it on `localhost:8501`
- ✓ Saw the 24-day roadmap and where today fits

CHECK YOUR LAPTOP NOW

```
# Should print a version 3.12+  $ python --version # Should see (.venv) in your prompt after activating (.venv) $ echo $VIRTUAL_ENV /path/to/your/.venv # Should open in browser (.venv) $ streamlit hello
```

⚒ IF ANY OF THESE FAILED — raise your hand now. We fix it in 5 minutes before we start today's material.

A function is a tiny contract: I give you this, you give me that.

In a textbook, functions are about "reusing code." In a real project, they're about hiding complexity behind a name you can trust.

BEFORE **WHAT YOU PROBABLY WROTE**

```
# The script-style approach name = input("Your name: ") print(f"Hello, {name}") # Works. But: # - hard to reuse with a different input source # - no way to test it without typing # - mixes "get data" with "show data"
```

Fine for a 10-line script. Falls apart at 100.

AFTER **A REAL FUNCTION**

```
def greet(name: str) -> str: """Return a greeting for the given name.""" return f"Hello, {name}" # Now you can: msg = greet("Aarav") # from a variable msg = greet(input("Name: ")) # from user input msg = greet(row[0]) # from a CSV row print(msg) # OR write to file # The function doesn't care WHERE name came from.
```

Same logic. Three different input sources. One function.

KEY IDEA A good function **takes data in**, **returns data out**, and doesn't care where the data came from or where it goes next.

Six parts. One function. Learn to read each one.

Once you can name every piece of a function, you can write one without copy-pasting from Stack Overflow.

```
①def clean_price(text: str) -> float:
②    """Parse a price string like 'Rs. 499' into a float."""
③    digits = ''.join(ch for ch in text if ch.isdigit() or ch == '.')
④    if not digits:
return 0.0
⑤    return float(digits)
⑥ price = clean_price('Rs. 499')
# price is now 499.0
```

LINE BY LINE

① **def keyword + name + signature**

Names the function. The signature says it takes one arg called `text` (a string) and returns a float.

② **docstring**

A triple-quoted string right after the def line. Tells other humans what this does. Tools like VS Code show it on hover.

③ **body — the actual logic**

Pulls out only digits and decimal points. Uses a generator expression inside `''.join(...)`. Real project code looks like this.

④ **guard clause**

If there are no digits at all, return 0.0 early. Prevents a ValueError on the next line.

⑤ **return statement**

Hands the value back to whoever called the function. Without return, you get None.

⑥ **calling the function**

How other code uses it. The return value lands in `price`.

Type hints (`: str`, `-> float`) are documentation, not enforcement. Python won't stop you passing an int — but your IDE will warn you, and other developers will thank you.

Three ways to hand data to a function.

You'll see all three in real codebases. Knowing which one you're looking at is half the battle.

POSITIONAL 01

```
def add(a, b):
return a + b
add(2, 3)     # → 5
add(3, 2)     # → 5
add(2)        # ✗ TypeError, missing b
```

Order matters. Most common form.

KEYWORD 02

```
def greet(name, greeting):
return f"{greeting}, {name}!"
greet(name="Aarav", greeting="Hi")
greet(greeting="Hi", name="Aarav")
# Both work — order doesn't
# matter when you use keywords.
```

Order-independent. Use when args are unclear.

DEFAULT 03

```
def greet(name, greeting="Hello"):
return f"{greeting}, {name}!"
greet("Aarav")              # →
Hello, Aarav!
greet("Aarav", "Yo")        # → Yo,
Aarav!
greet("Aarav", greeting="Hi")
```

Optional args with sensible defaults. Readable + flexible.

COMBINING THEM Positional first, then keyword, then defaults. Python enforces this order. `def f(a, b=2, c)` is a **SyntaxError**.

A function without `return` doesn't return nothing. It returns `None`.

This single fact causes more beginner bugs than any other. Once you see it, you can't unsee it.

✦ THE BUG

```
def greet(name):
print(f"Hello, {name}!")
# No return statement!
msg = greet("Aarav")
# → prints: Hello, Aarav!
print(msg)
# → prints: None
msg.upper()
# ✕ AttributeError: 'NoneType' object
#   has no attribute 'upper'
```

The function looked like it worked. Then the next line exploded.

✓ THE FIX

```
def greet(name) -> str:
return f"Hello, {name}!"
# return, not print
msg = greet("Aarav")
# msg is now "Hello, Aarav!"
print(msg)            # → Hello, Aarav!
print(msg.upper())    # → HELLO, AARAV!
print(msg + " 👋")    # works too
```

Now you can do anything with the result — print it, save it, pass it on.

RULE OF THUMB A function should either **RETURN** a value or **DO** something (print, write a file), rarely both. Pick one job per function.

EXERCISE · 5 MIN · TYPE ALONG

Write a function that extracts the year from a date string.

On your own laptop, in VS Code. Three minutes to try, two minutes to compare with your neighbour.

BRIEF

Write a function `extract_year(date_str)` that takes a date string like `"2026-07-29"` or `"29/07/2026"` and returns just the year as an integer.

REQUIREMENTS

- ✓ Function name must be `extract_year`
- ✓ Takes one argument (a string)
- ✓ Returns an `int`, not a string

EXAMPLES

```
extract_year("2026-07-29")     # → 2026
extract_year("29/07/2026")     # → 2026
extract_year("July 29, 2026") # → 2026
```

SAMPLE SOLUTION

```
def extract_year(date_str: str) -> int:
"""Pull the 4-digit year out of a date
string."""
digits = [ch for ch in date_str if
ch.isdigit()]
# For "2026-07-29"
# → ['2','0','2','6',
#    '0','7','2','9']
# Take the first 4 → year
year_str = ''.join(digits[:4])
return int(year_str)
# Try it:
print(extract_year("2026-07-29"))    # 2026
print(extract_year("29/07/2026"))    # 2026
print(extract_year("July 29, 2026")) # 2026
```

DON'T PEEK YET *Try it yourself first. The solution is here for after.*

PART 2 OF 3 · FILES & STORAGE

02 / 0

Files remember what *variables forget.*

Every project from Day 9 onward needs to save data between runs. We start the habit today.

01 OPEN & READ

02 WRITE & APPEND

03 WITH BLOCKS (CONTEXT MANAGERS)

CODE WALKTHROUGH · READING

Three ways to read a file. Pick the one that matches your data.

All three work. The right choice depends on whether you want one big string, a list of lines, or line-by-line processing.

WHOLE FILE AS STRING

01

```
withopen'poem.txt''r' as

# content is one big string
printlen              # total chars
print          '\n'  # line count
```

Small files. When you need everything at once.

ALL LINES AS LIST

02

```
withopen'poem.txt''r' as

# lines is a list of strings
# each ends with '\n' (except maybe the last)
for       in enumerate
   printf"{i}: {line.rstrip()}"
```

When you need line numbers or random access.

LINE BY LINE

03

```
withopen'poem.txt''r' as
   for    in

# Memory-friendly — only one line
# in memory at a time.
# Use this for big files.
```

Big files. When you process and forget.

! **ALWAYS USE `WITH` —** It auto-closes the file even if your code crashes. Without `with`, you have to remember `f.close()`, and you will forget.

Four modes. Pick by what you want to happen to the file.

Getting `'w'` vs `'a'` wrong is the most common way to lose data. Read this slide twice.

MODES CHEAT-SHEET

`'r'` READ
Open existing file for reading. Default. File must exist, else FileNotFoundError.

`'w'` WRITE
Create or OVERWRITE the file. Existing content is GONE. Use with care.

`'a'` APPEND
Create or ADD TO the end of the file. Existing content is preserved.

`'r+'` READ+WRITE
Open existing file for both. File must exist. You control the position.

⚠ **`'w'` IS DESTRUCTIVE —** If the file exists, `open("file.txt", "w")` wipes it instantly. No confirmation. No undo. Be sure before you run it.

WRITE & APPEND EXAMPLE

```
# WRITE — creates log.txt, or wipes it
with open 'log.txt' 'w' as
            "Session start\n"
            "Day 2 — Python, Properly\n"

# APPEND — adds to the end, keeps existing
with open 'log.txt' 'a' as
            "Closing exercise completed\n"
            f"Students present: 28\n"

# READ — verify what we wrote
with open 'log.txt' 'r' as
    print
# → Session start
#  Day 2 — Python, Properly
#   Closing exercise completed
#  Students present: 28
```

COMMON PITFALLS · 15 MIN

Five traps every Python beginner falls into. Once.

You will hit these. Best to see them on a slide first, so you recognise them in your own code later.

01 MUTABLE DEFAULT ARGS

```
# ✗ Bug — default list is shared!
def

   return

        # → [1]
        # → [1, 2] not [2]!
```

FIX — Use `lst=None` and create inside the function.

02 == VS IS

```
        # → True (same values)
 is     # → False (different objects)

# Use == for value comparison.
# Use is only for None, True, False.
```

FIX — Default to `==`. Reserve `is` for `is None` checks.

03 OFF-BY-ONE IN SLICING

```
        # → [20, 30]  (not 40!)
# → [10, 20, 30]
# → [40, 50]
# → 50  (last)
```

FIX — Slices are `[start:stop)`, stop is EXCLUSIVE.

04 STRING vs INT COMPARISON

```
    input"Age: "# "25" (string!)
if        # ✗ TypeError in Py3
   print"Adult"

# input() ALWAYS returns a string
    intinput"Age: "
if       # ✓ now works
   print"Adult"
```

FIX — Convert `input()` before comparing to numbers.

05 CLOSING A FILE TWICE

```
   open'data.txt'

# ... later ...
        # ✗ ValueError: I/O on closed file

# ✓ Use a with-block, this can't happen
withopen'data.txt' as

# f is auto-closed, you can't misuse it
```

FIX — Always use `with`. Manual `close()` is a footgun.

KEY TAKEAWAY

Every one of these is a one-line fix once you've seen it. Bookmark this slide.

You'll recognise each one in your own code within the week.

PREVIEW · ERROR HANDLING

When files go missing, your code shouldn't crash.

We'll go deeper on this later in the program. Today: just enough to read a file without your app exploding.

⊗ WITHOUT TRY/EXCEPT — crashes

```
# If the file doesn't exist, your
# program crashes immediately:
withopen'missing.txt'r' as


# ✕ FileNotFoundError: [Errno 2]
#   No such file or directory:
#   'missing.txt'
#
# Everything after this line never runs.
```

ⓘ *In a script, this is annoying. In a Streamlit app, this is a bad demo.*

WITH TRY/EXCEPT — recovers

```
try
    withopen'missing.txt'r' as

except
    # This block runs ONLY if the
    # specific error above happens
              ""
    print"File not found. Using empty string."
except
    # Different error, different handler
              ""
    print"No permission to read this file."
finally
    # Always runs, error or not
    printf"Content length: {len(content)}"
```

TODAY'S RULE — Always wrap `open()` in a try/except for `FileNotFoundError`. We'll cover the full error model in Week 3.

TOOLING · 5 MIN

Three ways to run a Python file in VS Code. Pick the one that fits.

You can run from the terminal like Day 1, but VS Code has built-in shortcuts worth knowing — especially when debugging.

01 ▶ THE PLAY BUTTON

▶ Ctrl+Enter Cmd+Enter

☞ HOW-TO

Open a `.py` file → click the ▶️ Play button in the top-right corner (or press `Ctrl+Enter` / `Cmd+Enter`).

◉ WHAT HAPPENS

VS Code runs the file in the integrated terminal at the bottom. Output appears there. You can type input too.

★ BEST FOR

Running a script end-to-end. The fastest option.

02 ✦ INTEGRATED TERMINAL

⌨ Ctrl+` Cmd+`

☞ HOW-TO

Open the terminal panel (`Ctrl+`` / `Cmd+``) → type `python filename.py` and hit Enter.

◉ WHAT HAPPENS

Same as running in a separate terminal window, but stays inside VS Code. Your venv is auto-activated if VS Code detected it.

★ BEST FOR

When you need to pass command-line arguments or pipe input.

03 ✦ DEBUG (F5)

✦ F5

☞ HOW-TO

Click to the left of a line number to set a red breakpoint → press `F5` (or use the Run → Start Debugging menu).

◉ WHAT HAPPENS

Code runs and PAUSES at your breakpoint. You can inspect variables, step line-by-line, and see the call stack.

★ BEST FOR

When a function returns the wrong thing and you don't know why. Worth learning now.

⚠ COMMON GOTCHA — If the Play button says "Select Python Interpreter" when you click it, pick the one that says `(.venv)` next to it. Otherwise VS Code uses your system Python and Streamlit won't be found.

03 /03

Build something with everything from today.

A function. A file. A return value. A real problem worth solving.

01 BRIEF (5 MIN) → 02 BUILD (15 MIN) → 03 REVIEW (5 MIN)

EXERCISE · BRIEF · 5 MIN

Read a messy contacts file. Return a clean list.

A real-world task — every CSV, every scraped list, every copy-pasted table starts messy. Your job: clean it.

THE BRIEF

Task

Write a function `clean_contacts(filepath)` that reads a text file of contacts and returns a list of `(name, email)` tuples. Skip lines that don't have an email.

REQUIREMENTS

- ✓ Function name must be `clean_contacts`
- ✓ Takes a filepath (string) as the only argument
- ✓ Returns a list of tuples, e.g. `[('Aarav', 'aarav@example.com'), ...]`
- ✓ Skip blank lines and lines without an `@` symbol
- ✓ Use a try/except in case the file is missing — return an empty list

BY THE END

Every student has a working `clean_contacts` function and can call it on a real file.

SAMPLE INPUT FILE

contacts.txt

```
# contacts.txt Aarav,aarav@example.com Priya,
priya@example.com Rahul,rahul@college.edu --- Meera,
Vikram,vikram@example.com extra
Sanjana,sanjana@example.com
```

EXPECTED OUTPUT

```
[('Aarav', 'aarav@example.com'), ('Priya', 'priya@example.com'), ('Rahul', 'rahul@college.edu'), ('Vikram',
'vikram@example.com extra'), ('Sanjana', 'sanjana@example.com')]
```

— five tuples. Blank lines, separator, and the Meera line without email all skipped.

EXERCISE · SOLUTION WALKTHROUGH

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

```python
①def clean_contacts(filepath: str) -> list:
"""Read a contacts file and return (name, email) tuples."""
②    try:
with open(filepath, 'r') as f:
lines = f.readlines()
except FileNotFoundError:
return []
③    contacts = []
for line in lines:
④        line = line.strip()
if not line:
continue
if line == '---':
continue
parts = line.split(',', 1)
if len(parts) != 2:
continue
⑤        name, email = parts[0].strip(), parts[1].strip()
if '@' not in email:
continue
⑥        contacts.append((name, email))
return contacts
# Try it:
result = clean_contacts('contacts.txt')
print(result)
# → [('Aarav', 'aarav@example.com'), ('Priya', 'priya@example.com'),
#    ('Rahul', 'rahul@college.edu'),
#    ('Vikram', 'vikram@example.com extra'),
#    ('Sanjana', 'sanjana@example.com')]
```

OTHER SOLUTIONS EXIST You might use a list comprehension, a regex, or pandas. All valid. This version is the most readable for a beginner.

LINE BY LINE — Walkthrough

1. **Signature + docstring**
   Takes a filepath string, returns a list. Docstring says what it does.
2. **try/except for missing file**
   If the file doesn't exist, return an empty list instead of crashing.
3. **Accumulator pattern**
   Start with an empty list, append to it as we find valid contacts.
4. **Clean + skip junk lines**
   Strip whitespace. Skip blank lines. Skip the `---` separator.
5. **Split + validate**
   Split on the first comma only (so emails with commas don't break). Skip lines without an `@`.
6. **Append as a tuple**
   Append `(name, email)` — note the parens make it a tuple, not two args.

NEXT SESSION · DAY 03

Git & GitHub.

Today we wrote code worth saving. Tomorrow we learn to save it like a team does.

Day 03

Git & GitHub

Goal — Nobody's afraid of Git anymore, and everyone has a GitHub account.

- ✦ **Why version control matters**
  Two people editing the same file, and the chaos that follows without Git.
- ↑ **Repos, commits, push and pull**
  Hands-on: create a GitHub account, make a repo, commit, push, pull.
- ✦ **Resolving a merge conflict**
  A deliberately created conflict, resolved together on screen.

BY THE END Every student has pushed at least one commit to their own GitHub repo.

BEFORE NEXT SESSION

Homework

- ☐ **Save your Day 2 work**
  Put `clean_contacts` and any other functions in a file called `day2.py`. You'll commit this on Day 3.
- ☐ **Create a GitHub account**
  If you haven't already. Use the same email as your Git config from Day 1.
- ☐ **Skim happygitwithr.com**
  Don't worry about the R parts — the GitHub account + install section is what you need.
- ☐ **Bring questions**
  Anything shaky from today's functions or file I/O. We open Day 3 with a 5-min Q&A.

Q&A · CLOSING · 15 MIN

Functions are verbs. Files are nouns.

You wrote both today. Tomorrow we put them under version control so they never get lost again.

OPEN THE FLOOR

Ask Now

- ✓ "Which function concept still feels shaky?"
- ✓ "Did `with open()` click, or does it feel like magic?"
- ✓ "Anything from the pitfalls slide you've hit before?"

BETWEEN SESSIONS

Get Unstuck

- ✓ Email · `hello@zlaark.com`
  For longer questions, code snippets welcome.
- ✓ Web · `www.zlaark.com`
  Program updates and resources.
- ✓ In-session · raise a hand
  No question is too small. We move at the room's pace.

RULES OF ENGAGEMENT

How We Work

- ✓ Min 80% attendance to qualify for the certificate.
- ✓ Save every day's code — Day 3 we start committing it.
- ✓ Demo Day is mandatory — every student presents.